

```
const St = imports.gi.St;
const Main = imports.ui.main;
const Tweener = imports.ui.tweener;

let text, button;

function _hideHello() {
    Main.uiGroup.remove_actor(text);
    text = null;
}

function _showHello() {
    if (!text) {
        text = new St.Label({ style_class: 'helloworld-label', text: "Hello, world!" });
        Main.uiGroup.add_actor(text);
    }

    text.opacity = 255;

    let monitor = Main.layoutManager.primaryMonitor;

    text.set_position(Math.floor(monitor.width / 2 - text.
width / 2),
                    Math.floor(monitor.height / 2 - text.
height / 2));

    Tweener.addTween(text,
        { opacity: 0,
          time: 2,
          transition: 'easeOutQuad',
          onComplete: _hideHello });
}

function init() {
    button = new St.Bin({ style_class: 'panel-button',
        reactive: true,
        can_focus: true,
        x_fill: true,
        y_fill: false,
        track_hover: true });

    let icon = new St.Icon({ icon_name: 'system-run',
        icon_type: St.IconType.SYMBOLIC,
        style_class: 'system-status-
icon' });

    button.set_child(icon);
    button.connect('button-press-event', _showHello);
}

function enable() {
    Main.panel._rightBox.insert_child_at_index(button, 0);
}
```



Figure 1: 'Hello world' extension

```
function disable() {
    Main.panel._rightBox.remove_child(button);
}
```

In the above code, *St* has been imported because that is the library that allows us to create UI elements. *Main* is the instance of a class that has all the UI elements, and we add all UI elements to this *main* instance. *Tweener* is responsible for handling all the animations that might be performed. Let's look at what each of the functions in the above code does.

- **Init()** - Create a button for the top panel by passing a map of properties from *St.bin*. In the map, set the button to be reactive to mouse clicks and set the style class of the button. Set it such that the button can be focused by keyboard navigation, and that the button only fills x-space and not y-space. Create an icon, and set the icon as a child of the button. Finally, bind the button press event, to call the function *_showHello*, which is responsible for displaying the 'Hello world' message.
- **Enable()** - Add the button we created earlier to the right panel of the top panel.
- **Disable()** - Remove the button from the right panel.
- Next, let us look at the *metadata.json* file:

```
{ "shell-version": ["3.4.1"], "uuid": "foo@bar", "name":
  "hello", "description": "OSFY extension" }
```

As you can see, in the above code, the *metadata.json* file stores data about your extension. Details stored by default are the shell-version for which the extension was created. (This value will vary depending on your version of the GNOME shell. Also, the syntax for extensions will be different if you are running a GNOME version before 3.2.) Other details stored in the *metadata.json* file are the name, UUID and description.

And in the end, let's look at *stylesheet.css*:

```
.helloworld-label {
    font-size: 36px;
    font-weight: bold;
```